

	UNIVERSIDAD DE LOS LLANOS		CÓDIGO: FO-DOC-112
	PROCESO GESTIÓN DE APOYO A LA ACADEMIA		VERSIÓN: 01 PÁGINA: 1 de 11
	FORMATO GUÍA PARA PRÁCTICAS DE LABORATORIO		FECHA: 02/09/2016
			VIGENCIA: 2016

LABORATORIO DE GRAFOS

UNIDAD ACADÉMICA: Ingeniería de Sistemas

CURSO: Estructuras de Datos

PRACTICA N° XX: 9

1. OBJETIVOS

- Comprender la representación de datos transaccionales mediante grafos.
- Construir y analizar un grafo de co-ocurrencia de productos a partir de un conjunto de datos de transacciones.
- Desarrollar una aplicación interactiva para la visualización y análisis de relaciones de compra.

2. CONSULTA PREVIA

Los temas esenciales para complementar esta práctica son:

- Recursividad.
- Programación Orientada a Objetos.
- Grafos.

3. FUNDAMENTO TEÓRICO

Un grafo es una estructura matemática que representa un conjunto de objetos (nodos o vértices) y las conexiones entre ellos (aristas o enlaces). En el contexto del análisis de transacciones comerciales, los nodos representan productos, y las aristas indican que dichos productos han sido comprados juntos en una misma transacción.

Conceptos Clave en el Análisis de Grafos

Nodos (Vértices):

Representan las entidades individuales, en este caso, los productos de la canasta básica presentes en las transacciones.

Aristas:

Conectan dos nodos si los productos correspondientes se han comprado juntos en al menos una transacción. Las aristas pueden tener pesos que indican la frecuencia de co-ocurrencia.

Grafo No Dirigido con Pesos:

En este laboratorio se utiliza un grafo no dirigido, ya que la relación entre productos es bidireccional (si el producto A se compra con B, la relación es la misma en sentido inverso). Los pesos de las aristas representan la cantidad de veces que dos productos se han comprado juntos.

Grado de un Vértice:

Es el número de aristas conectadas a un nodo. En este análisis, un grado más alto indica que un producto está asociado frecuentemente con otros productos, lo que puede reflejar su popularidad o centralidad en las transacciones.

Vecinos de un Vértice:

ELABORADO POR: Néstor Eduardo Suat Rojas	CARGO: Profesor Tiempo Completo	FECHA: 05/02/2025
--	---	-----------------------------

	UNIVERSIDAD DE LOS LLANOS		CÓDIGO: FO-DOC-112	
	PROCESO GESTIÓN DE APOYO A LA ACADEMIA		VERSIÓN: 01	PÁGINA: 2 de 11
	FORMATO GUÍA PARA PRÁCTICAS DE LABORATORIO		FECHA: 02/09/2016	
			VIGENCIA: 2016	

LABORATORIO DE GRAFOS

Son los nodos directamente conectados a un nodo específico. Analizar estos vecinos permite identificar productos complementarios o patrones de compra recurrentes.

Implementación Grafos

List.h

```
#ifndef GRAPH_LIST_H
#define GRAPH_LIST_H

#include <iostream>
#include <cassert>

using namespace std;

template<typename T>
struct Node{
    T data;
    Node<T>* next;
};

template<typename T>
class List {
private:
    Node<T>* begin;
    int count;
    Node<T>* makeNode(const T& value);
public:
    List();
    ~List();
    void insert(int pos, const T& value);
    void erase(int pos);
    T& get(int pos) const;
    void print() const;
    int size() const;
    Node<T>* search(const T& value);
};

#endif //GRAPH_LIST_H
```

List.cpp

```
#ifndef GRAPH_LIST_CPP
#define GRAPH_LIST_CPP
```

	UNIVERSIDAD DE LOS LLANOS		CÓDIGO: FO-DOC-112	
	PROCESO GESTIÓN DE APOYO A LA ACADEMIA		VERSIÓN: 01	PÁGINA: 3 de 11
	FORMATO GUÍA PARA PRÁCTICAS DE LABORATORIO		FECHA: 02/09/2016	
			VIGENCIA: 2016	

LABORATORIO DE GRAFOS

```

#include "List.h"

template<typename T>
List<T>::List(): begin(0), count(0){
}
template<typename T>
List<T>::~~List(){
    Node<T>* del = begin;
    while (begin){
        begin = begin->next;
        delete del;
        del = begin;
    }
}
template<typename T>
Node<T>* List<T>::makeNode(const T &value) {
    Node<T>* temp = new Node<T>;
    temp->data = value;
    temp->next = 0;
    return temp;
}
template<typename T>
void List<T>::insert(int pos, const T &value) {
    if(pos < 0 || pos>count){
        cout << "Error! The position is out of range." << endl;
        return;
    }
    Node<T>* add = makeNode(value);
    if(pos == 0){
        add->next = begin;
        begin = add;
    }else{
        Node<T>* cur = begin;
        for(int i=0; i<pos-1; i++){
            cur = cur->next;
        }
        add->next = cur->next;
        cur->next = add;
    }
    count++;
}

template<typename T>
void List<T>::erase(int pos) {
    if(pos < 0 || pos>count){
        cout << "Error! The position is out of range." << endl;
    }
}

```

ELABORADO POR: Néstor Eduardo Suat Rojas	CARGO: Profesor Tiempo Completo	FECHA: 05/02/2025
--	---	-----------------------------

	UNIVERSIDAD DE LOS LLANOS		CÓDIGO: FO-DOC-112	
	PROCESO GESTIÓN DE APOYO A LA ACADEMIA		VERSIÓN: 01	PÁGINA: 4 de 11
	FORMATO GUÍA PARA PRÁCTICAS DE LABORATORIO		FECHA: 02/09/2016	
			VIGENCIA: 2016	

LABORATORIO DE GRAFOS

```

        return;
    }
    if(pos == 0){
        Node<T>* del = begin;
        begin = begin->next;
        delete del;
    }else{
        Node<T>* cur = begin;
        for(int i=0; i<pos-1; i++){
            cur = cur->next;
        }
        Node<T>* del = cur->next;
        cur->next = del->next;
        delete del;
    }
    count--;
}

template<typename T>
T& List<T>::get(int pos) const{
    if(pos < 0 || pos>count-1){
        cout << "Error! The position is out of range." << endl;
        assert(false);
    }
    if(pos == 0){
        return begin->data;
    }else{
        Node<T>* cur = begin;
        for(int i=0; i<pos; i++){
            cur = cur->next;
        }
        return cur->data;
    }
}

template<typename T>
void List<T>::print() const{
    if(count == 0){
        cout << "List is empty." << endl;
        return;
    }
    Node<T>* cur = begin;
    while(cur){
        cout << cur->data;
        cur = cur->next;
    }
}

template<typename T>
int List<T>::size() const {
    return count;
}

```

	UNIVERSIDAD DE LOS LLANOS		CÓDIGO: FO-DOC-112
	PROCESO GESTIÓN DE APOYO A LA ACADEMIA		VERSIÓN: 01 PÁGINA: 5 de 11
	FORMATO GUÍA PARA PRÁCTICAS DE LABORATORIO		FECHA: 02/09/2016
			VIGENCIA: 2016

LABORATORIO DE GRAFOS

```
template<typename T>
Node<T> *List<T>::search(const T &value) {
    Node<T>* cur = begin;
    while(cur){
        if(cur == value) return cur;
        cur = cur->next;
    }
}

#endif
```

Graph.h

```
#ifndef GRAPH_GRAPH_H
#define GRAPH_GRAPH_H

#include <iostream>
#include "List.cpp"

using namespace std;

template<class T>
class Edge;
template<class T>
class Vertex;

template<class T>
class Edge{
public:
    Vertex<T>* to;
    int weight;
    friend ostream &operator<<(ostream &out, Edge<T>* edge) {
        out << "To: " << edge->to->data << ", Weight: " << edge->weight << endl;
        return out;
    }
};

template<class T>
class Vertex{
public:
    T data;
    int inDegree;
    int outDegree;
    List<Edge<T>*> connectedTo;
```

	UNIVERSIDAD DE LOS LLANOS		CÓDIGO: FO-DOC-112	
	PROCESO GESTIÓN DE APOYO A LA ACADEMIA		VERSIÓN: 01	PÁGINA: 6 de 11
	FORMATO GUÍA PARA PRÁCTICAS DE LABORATORIO		FECHA: 02/09/2016	
			VIGENCIA: 2016	

LABORATORIO DE GRAFOS

```

Vertex(const T& value);
~Vertex();
void addNeighbor(Vertex<T>* to, int weight=0);
int getWeight(const T& value);
friend ostream &operator<<(ostream &out, Vertex<T>* vertex) {
    out << vertex->data << endl;
    out << "In degree: " << vertex->inDegree << endl;
    out << "out degree: " << vertex->outDegree << endl;
    out << "Edges: " << endl;
    vertex->connectedTo.print();
    return out;
}
};

template<class T>
class Graph {
public:
    int count;
    List<Vertex<T>*> vertexList;
    Graph();
    ~Graph();
    Vertex<T>* addVertex(const T& value);
    Vertex<T>* getVertex(const T& value);
    void addEdge(const T& from, const T& to, int weight=0);
};

#endif //GRAPH_GRAPH_H

```

Graph.cpp

```

#include "Graph.h"

template<class T>
Vertex<T>::Vertex(const T& value) {
    data = value;
    inDegree = 0;
    outDegree = 0;
    connectedTo = {};
}

template<class T>
Vertex<T>::~~Vertex() {
}

template<class T>

```

	UNIVERSIDAD DE LOS LLANOS		CÓDIGO: FO-DOC-112	
	PROCESO GESTIÓN DE APOYO A LA ACADEMIA		VERSIÓN: 01	PÁGINA: 7 de 11
	FORMATO GUÍA PARA PRÁCTICAS DE LABORATORIO		FECHA: 02/09/2016	
			VIGENCIA: 2016	

LABORATORIO DE GRAFOS

```

void Vertex<T>::addNeighbor(Vertex<T> *to, int weight) {
    Edge<T>* temp = new Edge<T>;
    temp->to = to;
    temp->weight = weight;

    outDegree++;
    to->inDegree++;

    connectedTo.insert(connectedTo.size(), temp);
}

template<class T>
int Vertex<T>::getWeight(const T &value) {
    for(int i=0; i < connectedTo.size(); i++){
        Edge<T>* temp = connectedTo.get(i);
        if(temp->to->data == value){
            return connectedTo.get(i)->weight;
        }
    }
    return NULL;
}

template<class T>
Graph<T>::Graph() {
    count = 0;
    vertexList = {};
}

template<class T>
Graph<T>::~~Graph() {
}

template<class T>
Vertex<T>* Graph<T>::addVertex(const T &value) {
    Vertex<T>* newVertex = new Vertex<T>(value);
    vertexList.insert(vertexList.size(), newVertex);
    count++;
    return newVertex;
}

template<class T>
void Graph<T>::addEdge(const T& from, const T& to, int weight) {
    Vertex<T>* fromVertex = getVertex(from);
    if(!fromVertex){
        fromVertex = addVertex(from);
    }
    Vertex<T>* toVertex = getVertex(to);
    if(!toVertex){
        toVertex = addVertex(to);
    }
}

```

ELABORADO POR: Néstor Eduardo Suat Rojas	CARGO: Profesor Tiempo Completo	FECHA: 05/02/2025
--	---	-----------------------------

	UNIVERSIDAD DE LOS LLANOS		CÓDIGO: FO-DOC-112	
	PROCESO GESTIÓN DE APOYO A LA ACADEMIA		VERSIÓN: 01	PÁGINA: 8 de 11
	FORMATO GUÍA PARA PRÁCTICAS DE LABORATORIO		FECHA: 02/09/2016	
			VIGENCIA: 2016	

LABORATORIO DE GRAFOS

```

    }
    fromVertex->addNeighbor(toVertex, weight);
}

template<class T>
Vertex<T> *Graph<T>::getVertex(const T &value) {
    for(int i=0; i < vertexList.size(); i++){
        if(vertexList.get(i)->data == value) return vertexList.get(i);
    }
    return NULL;
}

```

main.cpp

```

#include <iostream>
#include "Graph.cpp"

using namespace std;
int main() {
    Graph<int> g;
    for(int i=0; i < 6; i++){
        g.addVertex(i);
    }

    g.addEdge(0,1,5);
    g.addEdge(0,5,2);
    g.addEdge(1,2,4);
    g.addEdge(2,3,9);
    g.addEdge(3,4,7);
    g.addEdge(3,5,3);
    g.addEdge(4,0,1);
    g.addEdge(5,4,8);
    g.addEdge(5,2,1);

    for(int vertexPos=0; vertexPos < g.vertexList.size(); vertexPos++){
        Vertex<int>* vertex = g.vertexList.get(vertexPos);
        for(int edgePos=0; edgePos < vertex->connectedTo.size(); edgePos++){
            Edge<int>* edge = vertex->connectedTo.get(edgePos);
            cout << "(" << vertex->data << ", " << edge->to->data << ", " <<
edge->weight << ")" << endl;
        }
    }

    cout << "Weight of Vertex 3 -> 5: " << g.getVertex(3)->getWeight(5) << endl;

    return 0;
}

```


	UNIVERSIDAD DE LOS LLANOS		CÓDIGO: FO-DOC-112	
			VERSIÓN: 01	PÁGINA: 9 de 11
	PROCESO GESTIÓN DE APOYO A LA ACADEMIA		FECHA: 02/09/2016	
	FORMATO GUÍA PARA PRÁCTICAS DE LABORATORIO		VIGENCIA: 2016	

LABORATORIO DE GRAFOS

}

4. EQUIPOS, MATERIALES Y REACTIVOS

Equipos	Materiales	Sustancias y/o Reactivos
Computadores	Software: IVisual Studio code	

5. PROCEDIMIENTO O METODOLOGÍA

Los pasos a seguir en la práctica son los siguientes:

- Integrantes: Trabajo en grupo (2 personas).
- Lugar: Sala de laboratorio.
- Tiempo de trabajo: 1 hora y 30 minutos de clase en el laboratorio.
- Actividad: Leer la guía del laboratorio, desarrollar la actividad propuesta durante la sesión en el laboratorio, presentar y sustentar la solución al docente. Si es necesario, subir la solución al medio designado para cada laboratorio.

6. RESULTADOS

Conjunto de Datos - Transacciones de Compra

El conjunto de datos de transacciones (disponible [aquí](#)) contiene información sobre transacciones de compra de productos de la canasta básica. Cada registro en el conjunto de datos representa una transacción única e incluye detalles como el ID de la transacción, el producto comprado, la cantidad adquirida y el precio unitario del producto.

El conjunto de datos está organizado en las siguientes columnas:

- **ID Transacción:** Un identificador único para cada transacción de compra.
- **Producto:** El nombre del producto comprado.
- **Cantidad:** La cantidad de unidades del producto adquiridas en la transacción.
- **Precio Unitario:** El precio individual de cada unidad del producto.

A continuación se muestra un ejemplo de los registros presentes en el conjunto de datos:

ID Transacción	Producto	Cantidad	Precio Unitario
----------------	----------	----------	-----------------

ELABORADO POR: Néstor Eduardo Suat Rojas	CARGO: Profesor Tiempo Completo	FECHA: 05/02/2025
--	---	-----------------------------

	UNIVERSIDAD DE LOS LLANOS	CÓDIGO: FO-DOC-112	
		VERSIÓN: 01	PÁGINA: 10 de 11
	PROCESO GESTIÓN DE APOYO A LA ACADEMIA	FECHA: 02/09/2016	
	FORMATO GUÍA PARA PRÁCTICAS DE LABORATORIO	VIGENCIA: 2016	

LABORATORIO DE GRAFOS

1	Arroz	2	5.99
1	Leche	1	2.49
1	Pan	3	1.99
2	Frijoles	4	3.99
2	Azúcar	1	4.29
3	Aceite	2	6.99
...	...	...	...

Uso del conjunto de datos

Este conjunto de datos puede ser utilizado para realizar análisis sobre los productos más populares, determinar las combinaciones de productos más comunes en las compras o evaluar los precios unitarios de los productos.

Para determinar los productos de mayor compra, se puede utilizar técnicas como la **construcción de un grafo de co-ocurrencia de productos**, en donde las conexiones entre los nodos representan las transacciones en las que se compraron dos productos juntos.

Construcción del grafo.

Los siguientes pasos permiten construir un grafo y analizarlo con el objetivo de identificar los productos comprados con mayor frecuencia en relación a otro producto.

1. **Preparación de los datos:** Utilizar el conjunto de datos de transacciones de compras que se mencionó previamente.
2. **Identificación de nodos:** Cada producto será representado como un vértice en el grafo. Hacer una lista de los productos únicos presentes en los datos. En el conjunto de datos de ejemplo, los productos únicos son: Arroz, Leche, Pan, Frijoles, Sal, Azúcar, entre otros.
3. **Asignación de pesos:** Asignar pesos a las aristas del grafo para representar la frecuencia de compra entre dos productos. Puede hacerlo contando el número de veces que dos productos aparecen juntos en una misma transacción. Si dos productos se compran juntos en una transacción, el peso de la arista que los conecta aumenta. Por ejemplo, si en una transacción se compran Arroz y Leche, el peso de la arista entre los vértices Arroz y Leche aumenta en 1.

ELABORADO POR: Néstor Eduardo Suat Rojas	CARGO: Profesor Tiempo Completo	FECHA: 05/02/2025
---	------------------------------------	----------------------

	UNIVERSIDAD DE LOS LLANOS	CÓDIGO: FO-DOC-112	
		VERSIÓN: 01	PÁGINA: 11 de 11
	PROCESO GESTIÓN DE APOYO A LA ACADEMIA	FECHA: 02/09/2016	
	FORMATO GUÍA PARA PRÁCTICAS DE LABORATORIO	VIGENCIA: 2016	

LABORATORIO DE GRAFOS

4. **Construcción del grafo:** Utilizar la lista de vértices y los pesos asignados para construir el grafo. Crear un vértice para cada producto y agregar aristas entre los vértices correspondientes según los pesos asignados.
5. **Análisis del grafo:** Realizar análisis para identificar los productos comprados con mayor frecuencia en relación a otro producto. Tener en cuenta las siguientes métricas para en análisis:
 - **Grado de vértice:** Calcular el grado de cada vértice, es decir, el número de aristas conectadas a ese vértice. Los vértices con un grado más alto indica productos que se compran con mayor frecuencia en general.
 - **Vecinos más cercanos:** Examinar los vecinos directos de un vértice en particular y observar cuáles tienen los pesos de arista más altos. Esto permite identificar los productos que se compran con mayor frecuencia en relación a un producto específico.

Desarrollo de aplicación

Realizar un menú de opciones de una aplicación que utiliza el grafo construido a partir de los datos de transacciones de compra:

1. **Mostrar productos disponibles:** Esta opción mostrará la lista de productos únicos presentes en los datos. Proporciona a los usuarios una visión general de los productos disponibles para su selección.
2. **Mostrar grafo de relaciones:** Esta opción mostraría el grafo construido a partir de los datos de transacciones de compra. Los usuarios podrían ver visualmente las conexiones entre los productos y cómo se relacionan entre sí.
3. **Mostrar productos con mayor frecuencia:** Esta opción mostrará los productos que se compran con mayor frecuencia en general, es decir, aquellos con un grado de nodo más alto en el grafo. Los usuarios podrían identificar los productos más populares y tener una idea de las preferencias generales de compra.
4. **Mostrar productos relacionados a un producto específico:** Esta opción permitiría a los usuarios seleccionar un producto y ver los productos que se compran con mayor frecuencia en relación a ese producto en particular. Podrían descubrir productos complementarios o relacionados que podrían interesarles.
5. **Salir de la aplicación:** Esta opción permitiría a los usuarios salir de la aplicación y finalizar su sesión.

7. BIBLIOGRAFIA

C++ Programming: An Object-Oriented Approach. McGraw-Hill Education (2019). Behrouz A. Forouzan, Richard Gilberg.

Data Structures: A Pseudocode Approach Using C (2004). Richard Gilberg, Behrouz A. Forouzan.

ELABORADO POR: Néstor Eduardo Suat Rojas	CARGO: Profesor Tiempo Completo	FECHA: 05/02/2025
---	------------------------------------	----------------------